Hierarchical Summarization with Oracle-Preserving Guarantees: Aligning Human Preferences with Data Extraction Pipelines

Mitchell Linegar

Abstract

Social scientists and engineers routinely process documents that exceed the context windows of current Large Language Models (LLMs) by splitting them into blocks, summarizing each, and merging the results up a tree. While ubiquitous, this hierarchical strategy acts as a black box: verifying the fidelity of the final summary against the full original text is often prohibitively expensive or impossible at scale. We propose a probabilistic framework to certify the quality of these pipelines without requiring end-to-end verification. We identify three local, auditable properties—leaf sufficiency, parentwise compatibility, and on-range stability—that act as semantic consistency checks on the atomic operations of the hierarchy.

Our main contribution is to show that these local checks are sufficient to guarantee global fidelity. By generalizing the theory of *mergeable summaries* (Agarwal et al., 2013) to non-commutative semantic tasks, we prove that if an LLM satisfies these properties locally, it preserves the task-relevant information (the "oracle" value) globally in expectation. This reduces the validation problem from reading the entire corpus to a constant-time statistical audit: sampling a small number of leaf and merge operations allows us to derive a rigorous upper bound on the deployment-level error. Furthermore, we show that these local properties effectively transport supervision: training preference models (like DPO) on summaries that pass this audit yields the same population optimum as training on the full text. The result is a methodology for building, auditing, and optimizing long-context pipelines that are mathematically grounded and practically scalable.

Motivation

Large corpora are indispensable in contemporary political science, but the objects we ultimately analyze are not the raw texts themselves. Researchers want structured signals: whether an article reports a protest event; which actor sponsored an amendment; the date and location of a strike; or the count of distinct demonstrations in a monthly newsletter. We formalize this by positing an *oracle* f^* that maps a full document x to the task value $f^*(x) \in \mathcal{Y}$ that the researcher actually cares about. In this view, LLMs are useful only insofar as they preserve—or at least accurately predict—the information contained in $f^*(x)$; summaries are valid research objects only if they leave the oracle untouched.

Two practical constraints make this nontrivial. First, many documents are longer than a model's context window. In practice, one partitions x into contiguous blocks, summarizes each block with an LLM, and recursively merges those summaries up a tree. Second, even when context is not the binding constraint, researchers often *choose* to summarize for cost, latency, or workflow reasons (e.g., to store compact "event summaries" and relabel them as coding rules evolve). Either way, hierarchical summarization changes the object of analysis. The core methodological question is therefore: *How can we guarantee this pipeline leaves $f^*(x)$ unchanged without running the expensive oracle on the full text?*

Contributions. Four claims structure the paper. First, we reinterpret mergeable sketches [Agarwal et al., 2013b] as semantic invariance conditions on text, with Appendix G showing that our local laws reduce to mergeability in the commutative case. Second, Section 2 describes the

algebraic "ideal world" in which summaries behave like homomorphisms on strings, giving us the normative target for pipeline design. Third, we introduce the three local laws (leaf sufficiency, parentwise compatibility, on-range stability) and treat them as prompt-engineering objectives that force a stochastic summarizer to act like a mergeable operator. Finally, Section D develops the probabilistic audit: a random search over the realized summary tree whose empirical failure rates become a deployment-level error budget. Together these pieces let us honor the new motivation while restoring the detail and specificity of the prior draft.

From Global Algebra to Local Sampling

Our answer relies on a shift in perspective: instead of trying to validate the final summary (which requires processing the full document to establish ground truth), we treat the summarization pipeline as a stochastic process and audit its atomic operations.

We introduce three local laws: *leaf sufficiency* (the summary preserves the oracle on small blocks), *parentwise compatibility* (merging two summaries yields the same oracle value as merging the underlying text), and *on-range stability* (re-summarizing a summary is inert). The laws can be satisfied by prompt design, DSPy-style inference-time optimization, or fine-tuning; the details of their structural implications live in Appendix B.

This approach can be viewed as a semantic generalization of *mergeable summaries* Agarwal et al. [2013b]. While that literature constructs algebraic data structures (sketches) to preserve commutative statistics (like counts or heavy hitters) under merge operations, we seek to preserve arbitrary semantic oracles f^* (like event extraction or narrative flow) under non-commutative string concatenation. We replace the explicit algebra of sketches with "learned" local consistency laws that force a generic summarizer to behave *as if* it were a mergeable sketch; Appendix G shows that when f^* is symmetric, our laws collapse exactly to the standard mergeability condition.

The power of this framework lies in its efficiency. Because we prove that local consistency implies global preservation (Theorem 1), we do not need to audit the root of the tree. Instead, we can perform a *probabilistic audit*: by sampling and verifying a fixed number of leaves and internal merges—operations that fit easily within a small context window—we can statistically bound the error of the entire pipeline. This "random search over the summary tree" is auditable: every checked node runs entirely within context, yet the resulting failure-rate estimates control the full-document error. The probabilistic audit thus decouples the cost of quality control from the length of the documents, making it possible to certify pipelines on book-length documents or year-long query logs using a constant sampling budget.

Optimizing for Consistency

Beyond auditing, these local laws provide a recipe for optimization. Since the conditions are local, they can serve as objective functions for prompt engineering or fine-tuning (e.g., via frameworks like DSPy). Rather than optimizing a model to "summarize well"—a vague objective—practitioners can optimize the model to satisfy parentwise compatibility: "ensure that $g(\text{summary}_A + \text{summary}_B)$ yields the same extracted events as $g(\text{text}_A + \text{text}_B)$." Because each constraint operates on a single edge of the tree, gradients or prompt-adjustment heuristics can be estimated via the same random sampling procedure used for audits.

Safe Learning on Summaries

Finally, this local view yields a clean connection to learning. We consider supervision that depends on documents only through their task value, a common factorization that covers proper scoring, coding tasks, and Bradley–Terry–Luce models for pairwise preferences. Under this

assumption, we show that training on oracle-preserving summaries is information-equivalent to training on full documents.

Specializing to Direct Preference Optimization (DPO), we prove that if the hierarchy passes our audit, training a policy on the summaries achieves the same population optimum as training on the originals. When preservation is only approximate, the gap in the DPO loss is quantitatively controlled by the error rates estimated in our audit. This connects the mechanics of scalable preprocessing to the objectives of alignment: if the pipeline respects the local laws, the preferences learned from the output are faithful to the source. Appendix E provides the full derivation and extends the argument to any supervision scheme that factors through f^*.

1 Connections to Related Work

Our focus on local merge compatibility draws on, and extends, several established theoretical traditions. We view our framework as a semantic generalization of randomized sketching, justified by decision-theoretic sufficiency.

Mergeable Summaries and Sketches. The closest formal analogue to our work is the theory of *mergeable summaries* for data streams [Agarwal et al., 2013a]. In that setting, a summary (or sketch) S supports a binary merge operator $\oplus$ such that $S(A \cup B) \approx S(A) \oplus S(B)$, with error guarantees that hold under arbitrary merge trees. Our framework can be understood as generalizing this logic to the semantic, non-commutative domain of natural language. Specifically, when the task oracle f^* is symmetric (e.g., word counts or bag-of-words statistics) and the metric is Euclidean, our *Parentwise Compatibility* law (L2) is mathematically isomorphic to the definition of a mergeable summary. In this specific case, checking L2 is equivalent to verifying that the LLM acts as a valid adder for a Count-Min sketch [Cormode and Muthukrishnan, 2005]. However, text data is rarely commutative—the order of paragraphs changes the meaning of a narrative. Our contribution is to impose the rigorous discipline of mergeable summaries on these non-commutative operations, replacing the explicit algebra of sketches with "learned" local consistency checks that force a stochastic summarizer to behave *as if* it were a mergeable data structure. Appendix G formalizes this bridge, mapping our notation $(\mathcal{X}, \oplus, f^*, g)$ onto the streaming literature and proving that L2 coincides with mergeability when the oracle is symmetric.

Decision Theory and Blackwell Sufficiency. A second foundation is decision theory. Under proper scoring rules, the optimal Bayesian action depends only on the posterior of the target; reporting the truth is optimal [Gneiting and Raftery, 2007, Dawid, 2007]. Our Assumption 1 instantiates this principle: if supervision factorizes through $f^*(X)$, then f^* is a *sufficient statistic* for the population decision problem. Replacing X with a summary that preserves f^* leaves the Bayes risk unchanged. This is a task-specific version of Blackwell sufficiency [Blackwell, 1953, Torgersen, 1991], where f^* acts as the sufficient reduction. While Blackwell's theorem is often invoked abstractly, our framework provides an operational audit to verify that a specific summarization pipeline actually retains this sufficiency in practice.

Preference Learning and RLHF. Modern alignment techniques, such as Direct Preference Optimization (DPO), model preferences using Bradley–Terry–Luce likelihoods driven by a reward function [Bradley and Terry, 1952, Rafailov et al., 2023]. A standard assumption in this literature is that human preferences depend on the input only through its task-relevant semantics (the oracle value), not incidental phrasing. Under this factorization, our preservation results imply that training DPO on oracle-preserving summaries attains the same population optimum as training on full documents. Absent such preservation, empirical performance gains

can be illusory: a learned policy might entangle with artifacts of a particular merge schedule—a failure mode our audit is specifically designed to detect.

Global Structure vs. Local Checks. Finally, we contrast our approach with corpus-level abstractions like Latent Dirichlet Allocation (LDA) or Structural Topic Models (STM) [Blei et al., 2003, Roberts et al., 2014]. While those methods discover latent global structures unsupervised, we assume a pre-defined outcome label f^* determined by the researcher. Furthermore, while our results induce a monoid congruence on strings when assumptions hold globally (discussed in Section F), we deliberately avoid assuming a global algebra. Instead, we rely on "local" consistency: checks enforced only on the specific leaves and merges realized by the scheduler. This allows us to certify the fidelity of a pipeline without requiring the oracle space $\mathcal{Y}$ to possess a strict algebraic structure that the task semantics may not justify.

In short, we borrow the algebraic intuition of mergeable sketches, adopt the sufficiency criteria of decision theory, and insist on edge-wise, testable equalities where practitioners actually operate. With this context in place, we turn to the formal setup.

2 The Mechanism: From Global Algebra to Local Consistency

To guarantee the fidelity of a hierarchical pipeline, we must first define what it means for a summary to be "correct" in a formal sense. We begin with the formal setup, describe the algebraic ideal we strive for, and then introduce the local consistency laws that effectively force a stochastic LLM to approximate that ideal.

2.1 Setup: Strings, Oracles, and Metrics

We model documents as finite strings of tokens in the free monoid $(\mathcal{X}, \oplus, \varnothing)$, where $\oplus$ denotes concatenation. The task of interest is represented by a fixed oracle $f^* : \mathcal{X} \to \mathcal{Y}$ that maps strings to task values (labels, structured tuples, embeddings, etc.). The quality of a summary is measured in the oracle space via a (pseudo)metric $d_{\mathcal{Y}} : \mathcal{Y} \times \mathcal{Y} \to [0, \infty)$.

A summarizer $g : \mathcal{X} \rightsquigarrow \mathcal{X}$ is a (possibly stochastic) map. When g is stochastic, we evaluate its fidelity in expectation. To keep notation minimal, we write $\mathbb{E}[\cdot]$ to average over the internal randomness of g (seeds, temperature). A summary z is considered a valid substitute for document x if they are indistinguishable under the oracle metric:

$$D(z, x) := d_{\mathcal{Y}}\big(f^*(z), f^*(x)\big) \approx 0.$$

Downstream learning signals only depend on the oracle. Let $S^\star : \mathcal{X} \times \mathcal{A} \to \mathbb{R}$ denote an arbitrary supervision signal (a score, reward, or loss) and let $\mathcal{A}$ be the action or label space. We assume this supervision factors through the task value.

Assumption 1 (Conditional factorization)**.** *There exists a measurable $\bar{Q} : \mathcal{Y} \times \mathcal{A} \to \mathbb{R}$ such that for every action $a \in \mathcal{A}$,*

$$\mathbb{E}[S^\star(X, a) \mid f^*(X)] = \bar{Q}(f^*(X), a) \quad \textit{almost surely.}$$

This assumption covers standard classification (the loss depends only on the latent label), preference learning (the reward depends on the task semantics), and noisy annotators (the conditioning is on X). Once $f^*(X)$ is known, there is no additional supervision signal left in the raw phrasing.

Definition 1 (Oracle sufficiency in expectation)**.** *The summarizer g is f^*-*sufficient in expectation *if $\mathbb{E}[d_{\mathcal{Y}}(f^*(g(x)), f^*(x))] = 0$ for all $x \in \mathcal{X}$.*

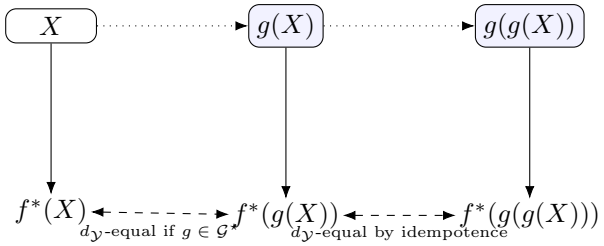

Figure 1: Oracle-preserving normal form. Sufficiency forces $f^*(g(X))$ to equal $f^*(X)$; on-range stability keeps additional summaries inert.

Definition 2 (Summary idempotence)**.** *The summarizer g is* summary-idempotent *if every string already on the range of g keeps its oracle under another call to g: for any random Z supported on* range(g)*,*

$$\mathbb{E}\big[d_{\mathcal{Y}}(f^*(g(Z)), f^*(Z)) \,\big|\, Z\big] = 0.$$

Definition 3 (Exact classes $\mathcal{G}^\star$ and $\mathcal{G}^\star_{\text{stab}}$)**.** *Let $\mathcal{G}^\star := \{g : d_{\mathcal{Y}}(f^*(g(X)), f^*(X)) = 0 \text{ a.s.}\}$ and $\mathcal{G}^\star_{\text{stab}} := \{g \in \mathcal{G}^\star : g$ also satisfies Definition 2$\}$.*

Doob–Dynkin factorization implies $g \in \mathcal{G}^\star$ iff the oracle can be recovered as a measurable function of $g(X)$; stability simply says re-applying g cannot change that oracle. Figure 1 visualizes the equalities enforced by these requirements.

2.2 Document Decomposition

Long documents routinely exceed the model's context window, so we cut x into contiguous blocks $(b_1, \ldots, b_k)$ and summarize them along a realized full binary tree T. Every leaf stores the raw substring b_i and its summary $g(b_i)$; each internal node u concatenates the realized child strings $S(u_L), S(u_R)$ and stores $S(u) = S(u_L) \oplus S(u_R)$ alongside the composed summary $g(u) = g(g(u_L) \oplus g(u_R))$. Traversing T bottom-up produces the one-pass output $Z^{(1)}(x) := g(\text{root}(T))$, and additional normalization rounds are defined by $Z^{(R+1)}(x) := g(Z^{(R)}(x))$. This "summarize, concatenate, summarize again" recipe is the exact pipeline we audit later.

2.3 The Theoretical Ideal: Text as Algebra

If we were summarizing simple data structures, the requirements for g would be straightforward. Consider the case where f^* is a count of keywords. This operator is a homomorphism from the free monoid of strings to the additive monoid of integers: $f^*(u \oplus v) = f^*(u) + f^*(v)$. To preserve this information, a summarizer g (acting as a "sketch") would need to support a merge operation that is homomorphic to addition. This is the essence of *mergeable summaries* in database theory [Agarwal et al., 2013a]:

$$f^*(A \cup B) \approx \text{Query}\big(\text{Merge}(S_A, S_B)\big).$$

For complex semantic tasks—event extraction, narrative summarization, or legal analysis—the merge operation is concatenation, which is non-commutative ($u \oplus v \neq v \oplus u$). However, the core requirement remains the same. We seek a global algebraic consistency where summarizing the parts and merging them yields the same task value as processing the whole. Ideally, we want a g such that for *any* partition u, v:

$$d_{\mathcal{Y}}\Big(f^*(u \oplus v),\ f^*\big(g(u) \oplus g(v)\big)\Big) = 0.$$

If this global property held, hierarchical summarization would be trivially safe. However, Large Language Models are not naturally associative algebraic operators. They are stochastic, sensitive to phrasing, and prone to "drifting" when context is segmented. We cannot assume this global structure exists; we must *enforce* it.

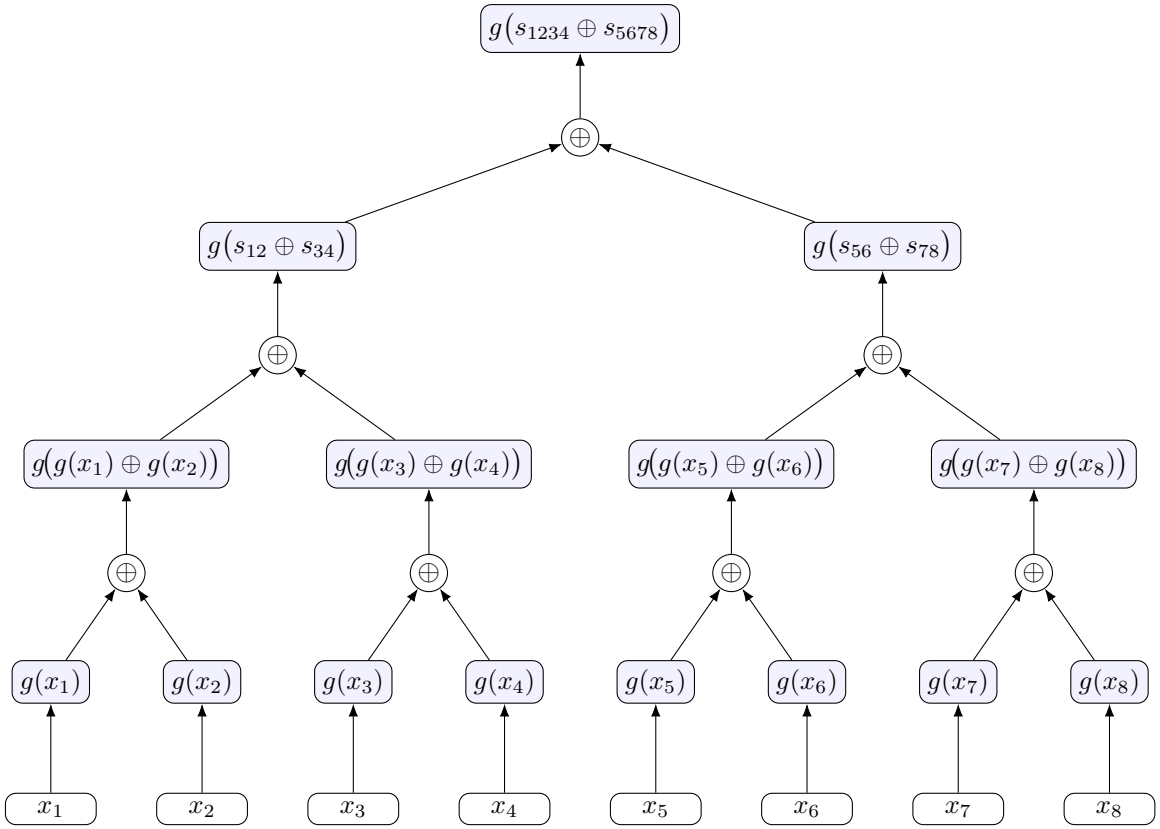

Figure 2: Hierarchical summarization as a binary tree. Leaves are summarized directly; their summaries are concatenated and re-summarized up the tree until a single root summary remains.

2.4 Local Consistency Laws

Since we cannot verify the global property on massive corpora (which would require running f^* on the full text, defeating the purpose of summarization), we focus on properties that are *local* and *auditable*.

We identify three laws that act as "prompt engineering objectives": if we can tune or prompt the model to satisfy these conditions on the atomic blocks it actually sees, the global guarantees follow automatically.

Convention 1 (Standing expectation convention)**.** *Expectations* $\mathbb{E}[\cdot]$ *at a node* u *are conditional on the realized subtrees below it. They average over the coin flips used to generate the summaries at* u *and its children.*

Law 1: Leaf Sufficiency (The Base Case). The first requirement is simple fidelity at the atomic level. Does summarizing a single block b preserve the oracle?

$$\mathbb{E}\big[\, d_{\mathcal{Y}}\big(f^*(g(b)),\, f^*(b)\big)\,\big] \;=\; 0 \quad \text{for every realized leaf } b. \tag{L1}$$

If an LLM cannot summarize a single paragraph without losing task-relevant information (e.g., dropping a name), the hierarchy fails immediately. Operationally we implement this law by spot-checking random leaves: auditors run the oracle on b and $g(b)$ directly, so any disagreement reveals a failure long before the merge logic is invoked. Leaf sufficiency therefore pins down the fidelity requirements that every prompt must satisfy before we can even talk about trees or random audits.

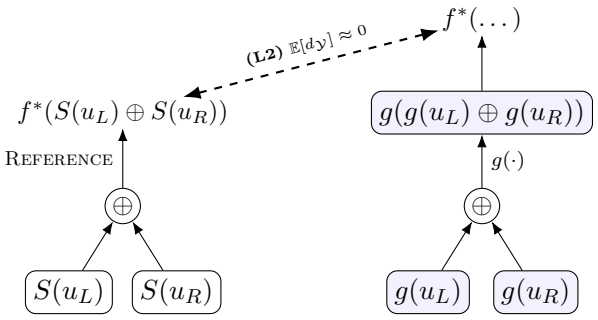

Figure 3: Parentwise Compatibility (L2) visualized. The left path shows the reference computation: concatenate the realized child texts and apply the oracle directly. The right path shows the hierarchical pipeline: reduce each child, concatenate the summaries, re-run the summarizer, and only then query the oracle. The dashed equality enforces that these two paths agree up to the task metric, so every merge behaves as if the summarizer were an associative algebraic operator.

Law 2: Parentwise Compatibility (The Inductive Step). This is the engine of our framework. At any internal merge node u with children u_L and u_R, there are two conceptually distinct ways to compute the oracle value. The *ground-truth route* concatenates the underlying text $S(u_L) \oplus S(u_R)$ and applies the oracle directly, while the *pipeline route* follows the actual system: summarize the children to obtain $g(u_L)$ and $g(u_R)$, concatenate those summaries, and summarize again to get $g(u)$ before invoking the oracle. Parentwise compatibility requires these two routes to agree in expectation (see Figure 3).

$$\mathbb{E}\Big[d_{\mathcal{Y}}\Big(\, f^*\big(g\big(g(u_L) \oplus g(u_R)\big)\big),\ f^*\big(S(u_L) \oplus S(u_R)\big)\,\Big)\Big] \;=\; 0. \tag{L2}$$

This law forces the LLM to behave *as if* it were an associative algebraic operator with respect to f^*. Note that this does not require the summary text to be identical to the source text, only that they yield the same task value. Figure 3 highlights where the tension lies: the dashed equality is where hallucinations or omissions surface, so each audit compares the two paths to ensure the merge step respects the oracle even though the inputs have already been compressed.

Remark 1 (Edge-wise compatibility)**.** *When an audit samples a realized edge (v, w) in the tree, it is often useful to probe both routes directly: evaluate $f^*(v \oplus w)$ once, evaluate $f^*(g(g(v) \oplus g(w)))$ once, and compare them under $d_{\mathcal{Y}}$. This pointwise check is simply the empirical version of L2 and offers fine-grained diagnostics about which merges violate the law.*

Law 3: On-Range Stability (Normalization). Finally, hierarchical pipelines often involve re-summarizing an output to smooth the style or reduce length. For this to be safe, the summarizer must be idempotent on its own range:

$$\mathbb{E}\big[\, d_{\mathcal{Y}}\big(f^*(g(Z)),\, f^*(Z)\big) \,\big|\, Z\big] \;=\; 0 \quad \text{for every realized summary } Z. \tag{L3}$$

Without this, repeatedly "cleaning up" a summary could cause the task information to drift (hallucination or forgetting). When L3 holds, any extra normalization pass—a style edit, a length cap, or a deduplication sweep—is provably inert at the oracle, so additional DSPy controllers or human edits cannot silently perturb f^*.

2.5 Consequences: Processing Invariance

The utility of these laws is that they allow us to transport trust from the leaves to the root. We do not need to verify the final summary against the whole book; we only need to verify that the "assembly line" is functioning correctly at each station.

Theorem 1 (Inductive Preservation)**.** *Fix a document partition $x = b_1 \oplus \ldots \oplus b_k$ and any full binary merge tree T. If L1 holds on every realized leaf and L2 holds at every realized internal node, then the final summary preserves the oracle in expectation:*

$$\mathbb{E}\left[d_{\mathcal{Y}}\big(f^*(Z^{(1)}), f^*(x)\big)\right] = 0.$$

Furthermore, if L3 holds, this preservation is maintained through any number of additional re-summarization rounds $Z^{(R)}$.

Proof. See Appendix B. The proof is a direct structural induction: L1 provides the base case, and L2 ensures the inductive step holds for the merge operation. □

Corollary 1 (Schedule invariance)**.** *For any fixed partition $(b_1, \ldots, b_k)$, every full binary tree on these leaves yields the same expected oracle whenever L1 and L2 hold on its realized edges. Balanced reductions and daisy chains are therefore interchangeable. (Proof in Appendix B.)*

Corollary 2 (Fold-of-folds invariance)**.** *Consider any two-level plan that first reduces contiguous "folds" and then reduces the fold summaries. If L1 and L2 hold on every realized edge and L3 holds on the intermediate summaries, the composite schedule preserves f^* in expectation regardless of the within-fold or over-fold parenthesizations. (Proof in Appendix B.)*

Together these results ensure that practitioners can pick whatever reduction schedule matches their infrastructure (balanced trees, daisy chains, fold-of-folds pipelines) without changing the expected oracle, provided the local laws hold in the realized hierarchy and L3 keeps additional normalization rounds inert.

By satisfying these local laws, we transform the summarizer g into a globally consistent operator. The remaining challenge is operational: how do we verify these laws hold without checking every single node?

2.6 Safe Learning on Summaries

Assumption 1 ensures that downstream training signals depend on documents only through $f^*(X)$. When a hierarchy satisfies L1–L3, the oracle of each realized node matches the oracle of its source text, so any supervision computed on top of the hierarchy is statistically indistinguishable from supervision computed on full documents. Appendix C formalizes this via Blackwell sufficiency, and Appendix E shows how it specializes to Direct Preference Optimization: once the audit certifies the tree (small p_{leaf}, p_{merge}, p_{stab}), training on summaries achieves the same optimum as training on full text up to a gap controlled by the measured distortion. This is the learning counterpart to the structural guarantees proven above.

3 The Probabilistic Audit: Certifying Pipelines at Scale

The theoretical guarantees in Section 2 and the learning equivalences in Section 2.6 rely on the assumption that the local laws hold. In a real deployment, we cannot simply assume these properties; we must verify them. However, checking every node in a massive tree defeats the purpose of summarization.

We solve this dilemma using a *probabilistic audit*. Because Theorem 1 guarantees that global fidelity is structurally implied by local fidelity, we do not need to validate the final summary against the full text. Instead, we treat the summarization pipeline as a stochastic process and estimate the failure rates of its atomic operations. This allows us to certify the quality of a corpus-scale dataset using a constant-time sampling budget.

3.1 The Audit Protocol

The audit estimates three quantities corresponding to the three local laws: the leaf failure rate (p_{leaf}), the merge failure rate (p_{merge}), and the stability failure rate (p_{stab}).

Sampling Leaves (Checking L1). We sample n_ℓ leaf blocks $\{b_i\}$ uniformly from the corpus. For each block, we generate its summary $g(b_i)$ and comparing the oracle values:

$$\hat{p}_{\text{leaf}} := \frac{1}{n_\ell} \sum_{i=1}^{n_\ell} \mathbb{I}\left\{ d_{\mathcal{Y}}\big(f^*(g(b_i)), f^*(b_i)\big) > \tau \right\},$$

where τ is a tolerance threshold (usually 0). This check is cheap because leaf blocks are by definition short enough to fit in the model context.

Sampling Merges (Checking L2). We sample n_m internal nodes from the realized reduction trees. For a sampled node u with children u_L and u_R, we must verify that the "pipeline route" matches the "ground truth route." Crucially, we do *not* need to expand the full subtree below u. We only need the *immediate* text associated with the children, $S(u_L)$ and $S(u_R)$. These are the inputs to the merge operation. The reference computation concatenates these texts and applies the oracle: $f^*(S(u_L) \oplus S(u_R))$. The candidate computation mirrors the pipeline by concatenating the child summaries, re-summarizing, and applying the oracle: $f^*(g(g(u_L) \oplus g(u_R)))$. The estimator is:

$$\hat{p}_{\text{merge}} := \frac{1}{n_m} \sum_{j=1}^{n_m} \mathbb{I}\left\{ d_{\mathcal{Y}}\big(\text{Reference}_j, \text{Candidate}_j\big) > \tau \right\}.$$

This check remains computationally feasible even high in the tree because we only process the immediate children, not the original raw text of the entire subtree.

Sampling Stability (Checking L3). If the pipeline includes post-processing rounds, we sample n_s summaries z_k that have already passed through g and check if re-summarization alters the oracle:

$$\hat{p}_{\text{stab}} := \frac{1}{n_s} \sum_{k=1}^{n_s} \mathbb{I}\left\{ d_{\mathcal{Y}}\big(f^*(g(z_k)), f^*(z_k)\big) > \tau \right\}.$$

3.2 The Deployment Error Budget

How do these local samples translate to a global guarantee? By applying the structural induction from Theorem 1, we can derive an upper bound on the expected error of the final root summary.

Consider a document split into N leaf blocks. A full binary merge tree will have $N - 1$ internal merge operations. If we perform $R - 1$ additional normalization rounds, the total accumulated error is bounded by the sum of errors at each step. We term this the **Deployment Error Budget**:

$$\Delta_{\text{total}} \leq \underbrace{N \cdot \hat{p}_{\text{leaf}}}_{\text{Leaf Errors}} + \underbrace{(N-1) \cdot \hat{p}_{\text{merge}}}_{\text{Merge Errors}} + \underbrace{(R-1) \cdot \hat{p}_{\text{stab}}}_{\text{Stability Errors}}. \tag{1}$$

This inequality provides a rigorous "worst-case" envelope. It assumes that errors accumulate additively (triangle inequality). In practice, errors often cancel out, making this a conservative safety certificate.

3.3 Connecting the Audit to Downstream Learning

The error budget Δ_{total} is not just a quality assurance metric; it directly quantifies the reliability of learning from summaries.

Recall from Section 2.6 that we wish to train a DPO policy π on summaries Z instead of documents X. The fundamental risk is that the policy will learn to exploit artifacts of the summarization rather than the true user preferences. We show in Appendix E that the gap between the policy learned on summaries ($\mathcal{L}^Z$) and the optimal policy learned on full text ($\mathcal{L}^X$) is Lipschitz-continuous with respect to the oracle distortion. Specifically, if the reward function is Lipschitz smooth with constant L_R, then:

$$|\mathcal{L}^X(\pi) - \mathcal{L}^Z(\pi)| \leq 2L_R \cdot \Delta_{\text{total}}.$$

This completes the link between the local audit and the final model performance. A low empirical failure rate in the audit ($\hat{p}_{\text{merge}} \approx 0$) mathematically guarantees that the preference signal has been transported intact to the summary.

3.4 Optimizing the Pipeline

Because the audit is local, it can be used as an optimization objective. If a pipeline fails the audit (e.g., $\hat{p}_{\text{merge}}$ is too high), practitioners can optimize the summarizer g specifically to minimize Equation L2 (Parentwise Compatibility). In practice this often looks like targeted prompt engineering where we explicitly instruct the model not to drop particular entity types during merges, DSPy-style controllers that treat the audit checks as metric assertions and automatically tune few-shot exemplars, or fine-tuning on synthetic (u_L, u_R) pairs so that the re-summarized candidate matches the reference oracle value. By optimizing for local consistency, we essentially force the LLM to respect the algebraic structure of the task, securing the global validity of the pipeline without ever needing to supervise the root.

4 Conclusion and Outlook

We have presented a framework to solve the central dilemma of long-document processing: how to scale semantic extraction to massive corpora without losing the ability to verify the results. By treating hierarchical summarization not as a black-box compression but as an algebraic reduction, we allow practitioners to audit the fidelity of the entire pipeline by inspecting only its atomic parts.

Our approach rests on a semantic generalization of *mergeable summaries*. While the database literature relies on explicit, commutative sketches to preserve counts and frequencies, we substitute these with "learned" local consistency laws—Leaf Sufficiency L1, Parentwise Compatibility L2, and On-Range Stability L3—that force stochastic LLMs to behave *as if* they were rigorous algebraic operators. This connects the flexibility of modern language models with the structural guarantees of randomized streaming algorithms.

The primary contribution is operational. We replace the impossible requirement of end-to-end verification (reading the whole library to check the summary) with a scalable *probabilistic audit*. By sampling realized leaves and internal merges, estimating their failure rates, and composing them via our structural theorems, practitioners can derive a rigorous **Deployment Error Budget**. This budget makes visible the precise trade-offs between block size, merge schedule, and accuracy, allowing social scientists to certify the reliability of their data extraction without needing to manually code a statistically significant fraction of the full corpus.

These guarantees extend beyond measurement to learning. We showed that under standard factorization assumptions (where supervision depends only on the oracle f^*), training Direct Preference Optimization (DPO) policies on validated summaries is information-equivalent

to training on full documents. The gap between the ideal policy and the learned policy is quantitatively controlled by the error budget derived from our audit. This ensures that alignment efforts are not fitting to artifacts of the summarization schedule, but are capturing the true underlying task semantics.

The limits of this approach are clear. The strongest equivalences hold only when supervision strictly factors through the task oracle; if human annotators reward presentational features lost during summarization, the alignment may drift. Furthermore, on-range stability is substantive: without it, additional "clean-up" rounds can silently flip labels, a failure mode our audit is designed to detect.

Methodologically, the blueprint is simple but demanding: Build hierarchies that pass the edge-wise audit; optimize models to satisfy local compatibility; and report certified error budgets alongside codebooks and regression tables. The result is a pipeline whose guarantees align with the research design, whose failures are interpretable, and whose outputs can be trusted to mean what the social scientist claims they mean.

References

Pankaj K. Agarwal, Graham Cormode, Zengfeng Huang, Jeff M. Phillips, Zhewei Wei, and Ke Yi. Mergeable summaries. *ACM Transactions on Database Systems*, 38(4):28:1–28:40, 2013a.

Pankaj K Agarwal, Graham Cormode, Zengfeng Huang, Jeff M Phillips, Zhewei Wei, and Ke Yi. Mergeable summaries. *ACM Transactions on Database Systems*, 38(4):1–28, 2013b.

David Blackwell. Equivalent comparisons of experiments. *Annals of Mathematical Statistics*, 24 (2):265–272, 1953.

David M. Blei, Andrew Y. Ng, and Michael I. Jordan. Latent dirichlet allocation. *Journal of Machine Learning Research*, 3:993–1022, 2003.

Ralph A. Bradley and Milton E. Terry. Rank analysis of incomplete block designs: I. the method of paired comparisons. *Biometrika*, 39(3/4):324–345, 1952.

Graham Cormode and S. Muthukrishnan. An improved data stream summary: The count-min sketch and its applications. In *Journal of Algorithms*, volume 55, pages 58–75, 2005.

A. P. Dawid. The geometry of proper scoring rules. *Annals of the Institute of Statistical Mathematics*, 59:77–93, 2007.

Tilmann Gneiting and Adrian E. Raftery. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477):359–378, 2007. doi: 10.1198/016214506000001437.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Margaret E. Roberts, Brandon M. Stewart, and Dustin Tingley. Structural topic models for open-ended survey responses. *American Journal of Political Science*, 58(4):1064–1082, 2014. doi: 10.1111/ajps.12103.

Erik Torgersen. *Comparison of Statistical Experiments*. Cambridge University Press, 1991.

This appendix provides complete technical foundations, detailed proofs, and practical guidance for the hierarchical summarization framework. It is organized to support both theoretical understanding and empirical deployment.

Appendix A: Notation Recap and Local Foundations collects all notation, definitions, and the foundational Lemma 1 used throughout. Readers comfortable with the main text may skip to Appendix B.
Appendix B: Proofs of Main Results establishes schedule invariance (Corollary 1), fold-of-folds invariance (Corollary 2), and multi-round preservation (Theorem 2)—the three core guarantees that make hierarchical processing practical.
Appendix C: Blackwell-Style Score Transport shows how supervision signals factor through the oracle f^* under Assumption 1, enabling training on summaries instead of full documents.
Appendix D: Practical Audit Guidance provides step-by-step audit procedures for deployed hierarchies, including sample-size calculations and methods for composing leaf-level and merge-level distortions into deployment-level bounds.
Appendix E: Direct Preference Optimization connects oracle preservation to preference learning, proving that DPO on summaries recovers the same optimal policies as DPO on full documents when the local laws hold.
Appendix F: Global Compatibility Variant presents an alternative formulation using global associativity and commutativity assumptions, connecting the framework to algebraic structures like monoid congruences and mergeable sketches.
Appendix H: Existence of Oracle-Preserving Summaries establishes that oracle-preserving summarizers always exist (set-theoretically) and can be implemented as compact encodings when the oracle space admits short representations.
Appendix I: Worked Examples instantiates the framework for binary event detection, entity extraction, counting aggregates, and provides a counterexample illustrating why stability cannot be omitted.

A Notation Recap and Local Foundations

This appendix collects the primitives needed before the first theorem in Section 2. It spells out every object once, keeping the notation flexible enough that we can later drop explicit "g" symbols and simply write g for the hierarchical evaluation.

A.1 Strings, oracle, and metric

We work on the free monoid $(\mathcal{X}, \oplus, \varnothing)$: $\mathcal{X}$ is the set of all finite token sequences, $\oplus$ is the usual concatenation of strings, and $\varnothing$ is the empty sequence acting as the identity. Documents are random variables $X \in \mathcal{X}$ drawn from a population law ρ. When we fix a specific document, we denote its base, unsummarized form by x_0 so that the realized leaf blocks are literally contiguous substrings of x_0. A fixed oracle $f^* : \mathcal{X} \to Y$ maps strings to their task value (labels, tuples, rewards, etc.). A measurable (pseudo)metric $d_{\mathcal{Y}} : Y \times Y \to [0, \infty)$ scores task disagreement, and we write the induced distortion as

$$D(z, x) := d_{\mathcal{Y}}\big(f^*(z), f^*(x)\big).$$

A.2 Summarizers and expectations

A summarizer is a (possibly stochastic) Markov kernel $g : \mathcal{X} \rightsquigarrow \mathcal{X}$ that turns any input string into a string-valued random output. Whenever randomness is present we average over the seeds or call counts used by g; when g is deterministic the equalities below hold pointwise. To avoid notational clutter we simply write $\mathbb{E}[\cdot]$ for all expectations and suppress the implicit conditioning on the realized substrings.

A.3 Partitions, trees, and hierarchical evaluation

Fix a document $x = b_1 \oplus \cdots \oplus b_k$; equivalently $x_0 = x$ and $b_1, \ldots, b_k$ are contiguous substrings drawn from x_0. A emphrealized partition and tree consists of those blocks together with a full binary tree T whose leaves, read left to right, are $(b_1, \ldots, b_k)$. Leaves therefore contain no summaries—only raw text—and internal nodes contain summaries with two ordered children denoted u_L, u_R. Each node $u \in T$ carries two associated strings:

$$S(u) = \begin{cases} b_i, & u \text{ is the } i\text{th leaf}, \\ S(u_L) \oplus S(u_R), & u \text{ internal with children } u_L, u_R, \end{cases}$$

and the hierarchical evaluation of g at u, denoted

$$g(u) := \begin{cases} g\big(S(u)\big), & u \text{ leaf}, \\ g\big(g(u_L) \oplus g(u_R)\big), & u \text{ internal}. \end{cases}$$

Summary of symbols and expectations. The following table collects the core notation used throughout the appendix and main text.

Symbol	Meaning
x_0	Base (unsummarized) document; each leaf block b_i is a contiguous substring
b	A realized leaf block input to g
u	Internal node with children u_L, u_R; raw text $S(u) = S(u_L) \oplus S(u_R)$
r	Root of tree T
$g(T')$	Summary at root of subtree T'; equivalently $g(T') = g(r')$ for root r' of T'
$Z^{(R)}(x)$	Rth-round summary: $Z^{(1)}(x) = g(r)$, $Z^{(R+1)}(x) = g(Z^{(R)}(x))$
$\mathbb{E}$	Expectation over randomness in g (pointwise if g deterministic)

Table 1: Core notation for hierarchical summarization.

A.4 Local laws

The guarantees in the main text follow from three statements checked *only* on the leaves and internal nodes that the realized tree touches. All three definitions coincide with their counterparts in Section 2; we restate them here to make the appendix self-contained.

Definition 4 (Leaf sufficiency (L1))**.** *For each realized leaf b, the single-block summary preserves the oracle in expectation:*

$$\mathbb{E}\big[d_{\mathcal{Y}}\big(f^*(g(b)), f^*(b)\big)\big] = 0.$$

This is Eq. (L1) *in the main text.*

Definition 5 (Parentwise compatibility (L2))**.** *For every realized internal node u with children u_L, u_R,*

$$\mathbb{E}\Big[d_{\mathcal{Y}}\Big(f^*\big(g(g(u_L) \oplus g(u_R))\big), f^*\big(S(u)\big)\Big)\Big] = 0.$$

The parent call thus agrees in expectation with the oracle on the underlying concatenation (Eq. (L2)*).*

Definition 6 (On-range stability (L3))**.** *For any random variable Z supported on the range of g,*

$$\mathbb{E}\big[d_{\mathcal{Y}}\big(f^*(g(Z)), f^*(Z)\big) \,\big|\, Z\big] = 0,$$

ensuring that re-summarizing a summary leaves the oracle unchanged (Eq. (L3)*).*

The edge-wise two-route comparison

$$d_{\mathcal{Y}}\Big(f^*(v \oplus w), f^*\big(g(g(v) \oplus g(w))\big)\Big) = 0$$

for realized child strings (v, w) is an optional audit probe (Remark 1).

Remark 2 (Task-level congruence)**.** *Whenever the distortion is zero, we may declare $x \sim x'$ if $d_{\mathcal{Y}}(f^*(x), f^*(x')) = 0$. This equivalence relation is a congruence on the free monoid (Appendix F) and motivates the view that g should output canonical representatives of task-equivalent strings.*

Notation snapshot.

A.5 Nodewise preservation and the first theorem

Lemma 1 (Nodewise preservation under the local laws)**.** *If L1 and L2 hold on all realized leaves and internal nodes of T, then for every node u,*

$$\mathbb{E}\Big[d_{\mathcal{Y}}\big(f^*(g(u)), f^*(S(u))\big)\Big] = 0.$$

Proof. If u is a leaf, $g(u) = g(S(u))$ and L1 yields the claim. If u is internal, expand $g(u) = g(g(u_L) \oplus g(u_R))$ and apply L2 with conditioning on the realized subtree. In either case the conditional expectation equals zero. □

Proof of Theorem 1. Apply Lemma 1 to the root r of T. The conditional expectation $\mathbb{E}_r$ coincides with $\mathbb{E}$ because every coin flip used anywhere in the reduction of T sits below r. Moreover $S(r) = x$, establishing the claim. □

This lemma–theorem pair captures everything needed before invoking more elaborate schedules (Corollary 1) or additional rounds. From this point forward we simply write $g(u)$ for the summary stored at node u.

B Proofs of Propositions and Theorems in the Main Text

This section collects the proofs of all corollaries and theorems stated in Section 2. These results establish three key invariance properties that make hierarchical processing practical: (i) schedule invariance—the expected oracle is the same whether we use balanced trees or daisy chains; (ii) fold-of-folds invariance—we can nest hierarchical reductions arbitrarily; and (iii) multi-round preservation—repeatedly "normalizing" summaries does not drift the oracle when on-range stability holds. All three follow from the local laws (L1)-(L3) via structural induction.

Throughout this section we fix a document $x = b_1 \oplus \cdots \oplus b_k \in \mathcal{X}$ together with a realized full binary reduction tree T on the leaves $(b_1, \ldots, b_k)$. For a node $u \in T$ we write $S(u)$ for the realized string at u and $g(u)$ for the corresponding hierarchical evaluation defined in Appendix A. Expectations with $\mathbb{E}$ average *only* over the internal randomness of g used in the computation of $g(u_L)$, $g(u_R)$, and the parent call at u, conditional on the realized subtree below u.

Every result below works solely with the task pseudometric $d_{\mathcal{Y}}$; equalities are always stated after applying f^*. Lemma 1 and Theorem 1 already establish nodewise and root preservation. Here we spell out three consequences: schedule invariance (any binary parenthesization on a fixed partition yields the same expected oracle), invariance under "fold-of-folds" execution plans, and multi-round preservation provided g is stable on its own range.

Corollary 3 (Schedule invariance)**.** *This restates Corollary 1 from the main text with full proof. The conclusion of Theorem 1 holds for every full binary tree T on the fixed leaf partition $(b_1, \ldots, b_k)$. In particular, for a given partition, balanced trees and daisy-chain schedules agree in expected oracle value.*

Proof. Lemma 1 applies to *any* realized node set. Changing the parenthesization merely changes which internal nodes appear, so the lemma re-runs verbatim and Theorem 1 yields the same root equality. □

Corollary 4 (Fold-of-folds invariance)**.** *This restates Corollary 2 from the main text with full proof. Let $(F_1, \dots, F_J)$ be a contiguous coarsening of the leaves. For each j, fix an arbitrary full binary within-fold tree T_j whose leaves, read left to right, concatenate to F_j. Let $\widehat{T}$ be any full binary over-fold tree whose leaves, read left to right, concatenate to $F_1 + \cdots + F_J$. Form the* composite tree T_{comp} *with root r by grafting $T_1, \dots, T_J$ into $\widehat{T}$ at its J leaves. If* (L1) *holds on the realized leaves of T_{comp} and* (L2) *holds at every realized internal node of T_{comp}, then*

$$\mathbb{E}\Big[\, d_{\mathcal{Y}}\big(f^*(g(r)),\ f^*(x)\big)\,\Big] \;=\; 0.$$

In particular, the expected oracle at the root is invariant to the choice of the within-fold parenthesizations and to the over-fold parenthesization, provided the realized leaves and internal nodes of T_{comp} satisfy (L1)–(L2)*. Moreover, if* (L3) *(on-range stability) also holds, then for the usual two-level "fold-of-folds" computation that first reduces each F_j to $S_j := g(T_j)$ and then reduces $(S_1, \dots, S_J)$ along $\widehat{T}$, we have*

$$\mathbb{E}\Big[\, d_{\mathcal{Y}}\big(f^*(g(\widehat{T})),\ f^*(x)\big)\,\Big] \;=\; 0,$$

because all additional g calls at the leaves of $\widehat{T}$ are re-applications of g to on-range strings and are inert at the oracle by (L3)*.*

Proof. Build the composite tree T_{comp} that first runs $T_1, \dots, T_J$ and then grafts their outputs into $\widehat{T}$. Every realized leaf or internal node across both levels appears exactly once inside T_{comp}, so Lemma 1 and Theorem 1 applied to T_{comp} give the displayed equality. For the two-stage implementation, note that the leaves of $\widehat{T}$ are already on the range of g, so on-range stability (L3) makes the extra applications of g inert in expectation. □

Theorem 2 (Multi-round preservation)**.** *This records the multi-round preservation result referenced in the main text. Assume Theorem 1 so that $\mathbb{E}[d_{\mathcal{Y}}(f^*(Z^{(1)}(x)), f^*(x))] = 0$. If, in addition, on-range stability* (L3) *holds, then the same equality holds for every $R \geq 1$ with $Z^{(R+1)}(x) := g(Z^{(R)}(x))$.*

Proof. Set $\Delta_R(x) := \mathbb{E}[d_{\mathcal{Y}}(f^*(Z^{(R)}(x)), f^*(x))]$. The base case $\Delta_1(x) = 0$ is Theorem 1. For the inductive step, use stability on the range of g to note that $\mathbb{E}[d_{\mathcal{Y}}(f^*(g(Z^{(R)}(x))), f^*(Z^{(R)}(x)))] = 0$. The triangle inequality thus gives $\Delta_{R+1}(x) \leq \Delta_R(x)$, and the sequence remains zero. □

C Blackwell–Style Score Transport

When the task involves learning from supervision—scores, rewards, or preferences—hierarchical summarization must preserve not just the oracle $f^*(X)$ but entire conditional score processes. This section establishes *score transport*: under Assumption 1, any supervision signal that factors through f^* can be transported from documents to summaries without information loss. The key insight is that sufficiency in the Blackwell sense (the summary's σ-field contains the oracle's) implies equality of all linearly aggregated expectations, including proper scoring rules and linear utilities. For nonlinear objectives such as logistic transforms in preference learning, exact preservation still holds when the local laws are satisfied exactly; approximate preservation is controlled by the oracle distortion Δ_R quantified in Appendix D.

We recall Assumption 1: there exists a measurable $\bar{Q} : \mathcal{Y} \times \mathcal{A} \to \mathbb{R}$ such that $\mathbb{E}[S^\star(X, a) \mid X] = \bar{Q}(f^*(X), a)$ for all $a \in \mathcal{A}$. The random object we compare across representations is always *real-valued*: either the conditional score process $a \mapsto \mathbb{E}[S^\star(X, a) \mid \cdot]$ or its linear images under bounded linear functionals Λ; we never take expectations of Y-valued quantities.

Exact score transport

The first statement is a Blackwell/Doob–Dynkin equality specialized to our setup. To avoid undefined expressions like $S^\star(Z, a)$, we define the summary-indexed score process by conditional expectation:

$$S^\star_Z(Z, a) \;:=\; \mathbb{E}\big[S^\star(X, a) \mid Z\big], \qquad a \in \mathcal{A}.$$

This averages the population score over all documents X that compress to the same summary Z. Intuitively, if the summary preserves all oracle information, then the score process on summaries must match the score process on documents.

We now show that when the summary is sufficient for the oracle—meaning $\sigma(f^*(X)) \subseteq \sigma(Z)$, i.e., the oracle can be recovered from Z—all expected scores are identical whether computed on full documents or on summaries.

Lemma 2 (Blackwell/Doob–Dynkin score transport)**.** *Assume Assumption 1. Let Z be any representation with $\sigma(f^*(X)) \subseteq \sigma(Z)$ (i.e., Z is sufficient for $f^*(X)$). Then for every $a \in \mathcal{A}$, the expected score is the same whether computed on documents or summaries:*

$$\mathbb{E}\big[S^\star(X, a)\big] \;=\; \mathbb{E}\big[S^\star_Z(Z, a)\big] \;=\; \mathbb{E}\big[\bar{Q}(f^*(X), a)\big].$$

Moreover, for any bounded linear functional Λ on the Banach space $(\mathcal{B}(\mathcal{A}), \|\cdot\|_\infty)$ of bounded real functions on $\mathcal{A}$,

$$\mathbb{E}\big[\Lambda\big(S^\star(X, \cdot)\big)\big] \;=\; \mathbb{E}\big[\Lambda\big(S^\star_Z(Z, \cdot)\big)\big] \;=\; \mathbb{E}\big[\Lambda\big(\bar{Q}(f^*(X), \cdot)\big)\big].$$

Proof. By the law of iterated expectations, $\mathbb{E}[S^\star(X, a)] = \mathbb{E}\{\mathbb{E}[S^\star(X, a) \mid X]\} = \mathbb{E}[\bar{Q}(f^*(X), a)]$. Also $\mathbb{E}[S^\star_Z(Z, a)] = \mathbb{E}\{\mathbb{E}[S^\star(X, a) \mid Z]\} = \mathbb{E}[S^\star(X, a)]$. Applying the bounded linear functional Λ and using linearity completes the proof. □

A particularly transparent special case is when Z *preserves the oracle exactly*, i.e., there exists a measurable h with $f^*(X) = h(Z)$ almost surely (Doob–Dynkin). Then $S^\star_Z(Z, a) = \mathbb{E}[S^\star(X, a) \mid Z] = \bar{Q}(h(Z), a)$, so for any bounded Λ,

$$\mathbb{E}\big[\Lambda\big(S^\star(X, \cdot)\big)\big] \;=\; \mathbb{E}\big[\Lambda\big(\bar{Q}(h(Z), \cdot)\big)\big].$$

In particular, when $Z = Z^{(R)}(X)$ is the R-round hierarchical reduction and the local laws hold *exactly* along the realized tree so that $f^*(Z^{(R)}(X)) = f^*(X)$ almost surely (Theorems 1 and 2), we may take $h(Z^{(R)}(X)) = f^*(Z^{(R)}(X))$ and obtain equality of all linearly aggregated supervision risks when training on X versus on $Z^{(R)}(X)$.

D Practical audit guidance

What matters at deployment is a single number,

$$\Delta_R \;:=\; \mathbb{E}\Big[\, d_{\mathcal{Y}}\big(f^*(Z^{(R)}(X)),\; f^*(X)\big)\,\Big],$$

the expected *task–space* distortion after one hierarchical pass and $R-1$ optional re-summarization rounds. All statements below assume $d_{\mathcal{Y}} \in [0, 1]$ (rescale if needed by $\mathrm{diam}(Y)$). The audit is local, deployment-specific, and piggybacks exactly on the two-route merge diagram: a parent can be reached either by "concatenate–summarize" or by "summarize children–concatenate–summarize again" (Figure 3).

Exact pass short-circuit. If the realized leaves satisfy (L1) exactly and the realized internal nodes satisfy (L2) exactly, Theorem 1 gives $\Delta_1 = 0$. If, in addition, (L3) holds, then Theorem 2 yields $\Delta_R = 0$ for all $R \geq 2$. In this regime there is nothing to bound; the audit reduces to checking that the empirical versions of the three quantities below are (within tolerance) zero.

What to estimate (local violation rates). Let a realized reduction tree have N leaves and $M = N - 1$ internal nodes. Denote realized leaves by b (each b is a contiguous substring of the base document x_0 and is never itself a summary), and realized internal nodes by u with children u_L, u_R, realized child strings $S(u_L), S(u_R)$, and realized child reductions $g(u_L), g(u_R)$. Define the *indicator* violations

$$p_{\text{leaf}} := \mathbb{E}\,\mathbf{1}\big\{d_{\mathcal{Y}}\big(f^*(g(B)),\, f^*(B)\big) > 0\big\},$$

$$p_{\text{merge}} := \mathbb{E}\,\mathbf{1}\Big\{d_{\mathcal{Y}}\Big(f^*\big(g(g(u_L) \oplus g(u_R))\big),\; f^*\big(S(u)\big)\Big) > 0\Big\},$$

$$p_{\text{idemp}} := \mathbb{E}\,\mathbf{1}\Big\{d_{\mathcal{Y}}\big(f^*(g(Z^{(1)}(x))),\, f^*(Z^{(1)}(x))\big) > 0\Big\} \qquad \text{(only if } R \geq 2\text{)}.$$

In code: log each raw leaf b (the substring of x_0 that entered the hierarchy), its summary $g(b)$, and, for every internal node, the pair $\big(S(u_L), S(u_R)\big)$ together with the resulting parent summary $g(u)$. Also record the seeds/call counts used by g so runs can be replayed. Then
emphsample leaves and internal nodes,
emphreplay the same randomness policy when needed, evaluate the displayed $d_{\mathcal{Y}}$-terms via f^*, threshold at > 0, and average. Conditioning on the realized subtree in the merge term matches (L2) exactly.

Crude but rigorous aggregation by union bound. Define the event that the root deviates after R rounds by

$$\mathcal{E}_R := \Big\{\; d_{\mathcal{Y}}\big(f^*(Z^{(R)}(X)),\; f^*(X)\big) > 0 \;\Big\}.$$

If no leaf fails L1 and no realized internal node fails L2 (and, for $R \geq 2$, no on-range re-summarization fails L3 at the root between rounds), then Theorems 1 and 2 force $\mathcal{E}_R$ not to occur. Therefore

$$\begin{aligned}\mathcal{E}_R \subseteq & \bigcup_{\text{leaves } b} \{d_{\mathcal{Y}}(f^*(g(b)), f^*(b)) > 0\} \\ & \cup \bigcup_{\text{internal } u} \Big\{d_{\mathcal{Y}}\Big(f^*\big(g(g(u_L)+g(u_R))\big),\; f^*\big(S(u)\big)\Big) > 0\Big\} \\ & \cup \bigcup_{t=1}^{R-1} \Big\{d_{\mathcal{Y}}\big(f^*(g(Z^{(t)}(X))), f^*(Z^{(t)}(X))\big) > 0\Big\}.\end{aligned}$$

Taking probabilities and applying the union bound gives the *crude envelope*

$$\begin{aligned}\Pr(\mathcal{E}_1) &\leq N\, p_{\text{leaf}} + M\, p_{\text{merge}}, \\ \Pr(\mathcal{E}_R) &\leq N\, p_{\text{leaf}} + M\, p_{\text{merge}} + (R-1)\, p_{\text{idemp}} \quad (R \geq 2).\end{aligned}$$

Since $d_{\mathcal{Y}} \in [0, 1]$, we have $\Delta_R = \mathbb{E}[d_{\mathcal{Y}}(\cdot, \cdot)] \leq \Pr(\mathcal{E}_R)$, hence

$$\Delta_1 \;\leq\; N\, p_{\text{leaf}} \;+\; M\, p_{\text{merge}}, \qquad \Delta_R \;\leq\; N\, p_{\text{leaf}} \;+\; M\, p_{\text{merge}} \;+\; (R-1)\, p_{\text{idemp}} \quad (R \geq 2). \tag{2}$$

These bounds hold with no assumption beyond the local laws used by Theorems 1 and 2. They are intentionally coarse: any single local failure is allowed to account for a root deviation. When (L3) holds, $p_{\text{idemp}} = 0$ and the multi-round term vanishes.

Plug-in estimates. In deployment we report the sample plug-ins

$$\widehat{p}_{\text{leaf}} := \frac{1}{n_\ell} \sum_{i=1}^{n_\ell} \mathbf{1}\big\{ d_{\mathcal{Y}}\big(f^*(g(b_i)), f^*(b_i)\big) > 0 \big\},$$

$$\widehat{p}_{\text{merge}} := \frac{1}{n_m} \sum_{j=1}^{n_m} \mathbf{1}\Big\{ d_{\mathcal{Y}}\Big(f^*\big(g(g(u_{L,j}) + g(u_{R,j}))\big),\ f^*\big(S(u_j)\big)\Big) > 0 \Big\},$$

and, if $R \geq 2$,

$$\widehat{p}_{\text{idemp}} := \frac{1}{n_r} \sum_{k=1}^{n_r} \mathbf{1}\Big\{ d_{\mathcal{Y}}\big(f^*(g(Z^{(1)}(x_k))),\ f^*(Z^{(1)}(x_k))\big) > 0 \Big\}.$$

The *crude deployment-level bound* is then

$$\widehat{\Delta}_1^{\text{ub}} := N\, \widehat{p}_{\text{leaf}} + M\, \widehat{p}_{\text{merge}},$$
$$\widehat{\Delta}_R^{\text{ub}} := N\, \widehat{p}_{\text{leaf}} + M\, \widehat{p}_{\text{merge}} + (R-1)\, \widehat{p}_{\text{idemp}} \quad (R \geq 2).$$

truncated at 1 if desired. Sampling should be random over the realized leaves and realized internal nodes of the run; if worried about depth or heterogeneity, stratify by depth or by child summary length and then aggregate. Labeling is whatever f^* requires (scripted or human).

Direct root estimation (often simplest). When you do not need a decomposition, estimate Δ_R directly by sampling documents x_i and computing $\widehat{\Delta}_R = \frac{1}{n} \sum_i d_{\mathcal{Y}}(f^*(Z^{(R)}(x_i)), f^*(x_i))$. If $R \geq 2$ and (L3) holds, $\Delta_R = \Delta_1$, so it suffices to measure $R = 1$.

E Direct Preference Optimization (DPO): Equivalence and Gap Bounds

Direct preference optimization trains policies by comparing preferred ("win") vs. dispreferred ("loss") responses. This section shows when training on hierarchically summarized documents $Z^{(R)}(X)$ yields the same optimal policies as training on full documents X. Under exact oracle preservation—when the local laws (L1)-(L3) hold exactly along the realized tree—the DPO objective depends on the input only through $f^*(\cdot)$, so the sets of population minimizers coincide. When preservation is approximate, we control the population gap by the oracle distortion Δ_R and provide explicit bounds under Lipschitz or bounded-reward assumptions. The key requirement is that annotator preferences factor through the oracle; if preferences depend on presentational features not encoded by f^*, training on summaries can deviate even when oracle preservation holds.

Let preferences follow Bradley–Terry–Luce with per-oracle rewards $\{R_y : \mathcal{A} \to \mathbb{R}\}_{y \in \mathcal{Y}}$ and scale $\beta > 0$:

$$\Pr\{a^w \succ a^\ell \mid X = x\} = \sigma\Big(\beta\, [R_{f^*(x)}(a^w) - R_{f^*(x)}(a^\ell)]\Big), \qquad \sigma(t) = \frac{1}{1+e^{-t}}.$$

DPO reparameterizes the reward through the policy π relative to a reference policy π_{ref} and minimizes

$$\mathcal{L}_{\text{DPO}}(\pi; \pi_{\text{ref}}) = -\,\mathbb{E}_{(X, a^w, a^\ell)} \left[\log \sigma\Big(\beta \Big\{ \log \frac{\pi(a^w \mid X)}{\pi_{\text{ref}}(a^w \mid X)} - \log \frac{\pi(a^\ell \mid X)}{\pi_{\text{ref}}(a^\ell \mid X)} \Big\} \Big) \right].$$

The log-ratio difference $\log \frac{\pi(a^w|X)}{\pi_{\text{ref}}(a^w|X)} - \log \frac{\pi(a^\ell|X)}{\pi_{\text{ref}}(a^\ell|X)}$ measures how much more the policy π favors the winner over the loser, relative to the reference. DPO trains π to maximize this gap for preferred pairs.

Definition 7 (Oracle-measurable policies)**.** *A policy π (and π_{ref}) is oracle-measurable if $\pi(\cdot \mid x) = \pi(\cdot \mid x')$ whenever $d_{\mathcal{Y}}\big(f^*(x), f^*(x')\big) = 0$.*

Theorem 3 (Exact DPO equivalence under oracle preservation and oracle–indexed pairs)**.** *Assume $d_{\mathcal{Y}}\big(f^*(Z^{(R)}(X)), f^*(X)\big) = 0$ almost surely for some $R \geq 1$. Assume also that the pair generator depends on X only through $f^*(X)$ and that π_{ref} is oracle-measurable with full support. Then the sets of population minimizers of $\mathcal{L}_{\mathrm{DPO}}$ coincide when trained on X and when trained on $Z^{(R)}(X)$. Moreover, there exists a population minimizer that is oracle-measurable; equivalently, we may without loss of generality restrict attention to oracle-measurable policies.*

Proof. Write $Y = f^*(X)$ and note that exact preservation implies $f^*(Z^{(R)}(X)) = Y$ almost surely. Under the oracle-indexed pair generator and oracle-measurable π_{ref}, the joint law of (Y, a^w, a^ℓ) is the same whether the input to the learning objective is X or $Z^{(R)}(X)$, and the two inner logits depend on the input only through Y. Therefore $\mathcal{L}_{\mathrm{DPO}}$ decomposes as an expectation over Y of a functional that depends on $\pi(\cdot \mid \cdot)$ only through its Y–sections. For each y, any Bayes–optimal $\pi(\cdot \mid y)$ solves an identical one-dimensional problem under X and under $Z^{(R)}(X)$, so the sets of minimizers coincide and every minimizer is Y–measurable (oracle-measurable). □

Theorem 4 (Quantitative DPO gap under metric distortion)**.** *Let $\Delta_R = \mathbb{E}\big[\, d_{\mathcal{Y}}\big(f^*(Z^{(R)}(X)), f^*(X)\big)\,\big]$ be the expected oracle distortion. To bound the population gap quantitatively, we impose regularity conditions ensuring that policies do not react radically to small changes in the oracle value. One option is a* policy-Lipschitz log-ratio *condition: there exists $L_\pi < \infty$ such that*

$$\left| \log \frac{\pi(a \mid y)}{\pi_{\mathrm{ref}}(a \mid y)} - \log \frac{\pi(a \mid y')}{\pi_{\mathrm{ref}}(a \mid y')} \right| \leq L_\pi \, d_{\mathcal{Y}}(y, y') \quad \forall\, a, y, y',$$

so the policy's preference for action a changes smoothly with the oracle value. Alternatively, we may assume a reward-Lipschitz exponential family*: $\pi(a \mid y) \propto \pi_{\mathrm{ref}}(a \mid y) \exp\{\beta^{-1} R_y(a)\}$ with $\sup_a |R_y(a) - R_{y'}(a)| \leq L_R \, d_{\mathcal{Y}}(y, y')$ for all y, y', which says the induced rewards vary smoothly in the oracle space. Then, writing $\mathcal{L}^X$ and $\mathcal{L}^Z$ for the DPO loss with inputs X and $Z^{(R)}(X)$, respectively,*

$$\big|\mathcal{L}^X(\pi) - \mathcal{L}^Z(\pi)\big| \;\leq\; \begin{cases} 2\,\beta\, L_\pi\, \Delta_R, & \textit{case (1)}, \\ 2\, L_R\, \Delta_R, & \textit{case (2)}, \end{cases} \qquad \textit{and} \qquad \inf_\pi \mathcal{L}^X(\pi) - \inf_\pi \mathcal{L}^Z(\pi) \;\leq\; \textit{same bound.}$$

Proof. Let $\varphi(t) = -\log \sigma(t)$, which is 1–Lipschitz. Denote the DPO logit by

$$\Lambda(X; a^w, a^\ell) \;:=\; \beta \left[\log \frac{\pi(a^w \mid X)}{\pi_{\mathrm{ref}}(a^w \mid X)} - \log \frac{\pi(a^\ell \mid X)}{\pi_{\mathrm{ref}}(a^\ell \mid X)} \right].$$

Then

$$\big|\mathcal{L}^X(\pi) - \mathcal{L}^Z(\pi)\big| \;\leq\; \mathbb{E}\big|\varphi(\Lambda(X; a^w, a^\ell)) - \varphi(\Lambda(Z^{(R)}(X); a^w, a^\ell))\big| \;\leq\; \mathbb{E}\big|\Lambda(X; a^w, a^\ell) - \Lambda(Z^{(R)}(X); a^w, a^\ell)\big|.$$

In case (1), oracle-measurability lets us replace X by $y = f^*(X)$ and $Z^{(R)}(X)$ by $y' = f^*(Z^{(R)}(X))$, and the Lipschitz condition yields

$$\big|\Lambda(X; a^w, a^\ell) - \Lambda(Z^{(R)}(X); a^w, a^\ell)\big| \;\leq\; \beta\, L_\pi \big(d_{\mathcal{Y}}(y, y') + d_{\mathcal{Y}}(y, y')\big) \;=\; 2\,\beta\, L_\pi\, d_{\mathcal{Y}}(y, y').$$

Taking expectations gives the stated $2\,\beta\, L_\pi\, \Delta_R$ bound. In case (2), write the logit difference as $\beta^{-1}\big[(R_y(a^w) - R_{y'}(a^w)) - (R_y(a^\ell) - R_{y'}(a^\ell))\big]$ and apply the reward-Lipschitz bound twice to get $2\, L_R\, d_{\mathcal{Y}}(y, y')$; take expectations to conclude. □

Remarks. (i) Because $d_{\mathcal{Y}} \geq 0$, any display of the form $\mathbb{E}[d_{\mathcal{Y}}(\cdot,\cdot)] = 0$ immediately implies $d_{\mathcal{Y}}(\cdot,\cdot) = 0$ almost surely with respect to the summarizer's randomness, conditional on the realized subtree. We use the stronger almost-sure formulation in the exact-equivalence statements for clarity. (ii) If π_{ref} is *not* oracle-measurable, exact preservation of f^* alone does not guarantee equivalence of minimizers; the KL term can transmit presentational artifacts of X into the optimizer. This is why the oracle-indexed pair generator and oracle-measurable π_{ref} hypothesis appears in Theorem 3.

F Global Compatibility Variant and Algebraic Structure

This section records a *global* counterpart to the local, auditable laws used in the main text. The point is conceptual: if the two-route identities were to hold *for all* pairs in $\mathcal{X} \times \mathcal{X}$, and if on-range parent calls depended only on child oracle values, then concatenation induces a *merge law on oracle values*. All equalities below are stated in the task metric $d_{\mathcal{Y}}$: when we write that two oracle outputs are "equal," this always means their $d_{\mathcal{Y}}$–distance is zero.

Remark 3 (How to read "equal at the oracle")**.** *We will say y and y' in $\mathcal{Y}$ are* equal at the oracle *if $d_{\mathcal{Y}}(y, y') = 0$. This can be thought of as identifying outcomes that the task metric cannot distinguish. More formally, this is the same as passing to the equivalence relation $y \sim y'$ iff $d_{\mathcal{Y}}(y, y') = 0$ and reading all statements modulo $\sim$.*

Global laws. We fix a *deterministic* summarizer $g : \mathcal{X} \to \mathcal{X}$ and assume three global properties.

Assumption 2 (Global f^*–sufficiency)**.** *For every $z \in \mathcal{X}$,*

$$d_{\mathcal{Y}}\big(f^*(g(z)),\ f^*(z)\big) = 0.$$

Assumption 3 (Global two-route compatibility)**.** *For every $u, v \in \mathcal{X}$,*

$$d_{\mathcal{Y}}\Big(f^*(u \oplus v),\ \ f^*\big(g\big(g(u) \oplus g(v)\big)\big)\Big) = 0.$$

Assumption 4 (On-range merge depends only on child oracles)**.** *There exists a measurable map $M : \mathcal{Y} \times \mathcal{Y} \to \mathcal{Y}$ such that for all $u, v \in \mathcal{X}$,*

$$d_{\mathcal{Y}}\Big(f^*\big(g\big(g(u) \oplus g(v)\big)\big),\ \ M\big(f^*(g(u)),\ f^*(g(v))\big)\Big) = 0,$$

and M is well-defined at the oracle level*: whenever $d_{\mathcal{Y}}(y_i, y_i') = 0$ for $i = 1, 2$, then*

$$d_{\mathcal{Y}}\big(M(y_1, y_2),\ M(y_1', y_2')\big) = 0.$$

The first law says a one-pass summary preserves the task value (globally, not just on realized leaves). The second is the global version of the edgewise two-route equality, with the *parent* summary explicit. The third law formalizes that, once inputs are on the range of g, the parent's oracle depends only on the two child oracle values; it also says this dependence respects "equality at the oracle" (outputs cannot change when inputs are changed within $d_{\mathcal{Y}}$–zero).

We now state two consequences.

Proposition 1 (Concatenation is a congruence for oracle equality on strings)**.** *If $a, a', b, b' \in \mathcal{X}$ satisfy $d_{\mathcal{Y}}\big(f^*(a), f^*(a')\big) = 0$ and $d_{\mathcal{Y}}\big(f^*(b), f^*(b')\big) = 0$, then*

$$d_{\mathcal{Y}}\big(f^*(a \oplus b),\ \ f^*(a' \oplus b')\big) = 0.$$

Proof. Let $U := g\big(g(a) \oplus g(b)\big)$ and $U' := g\big(g(a') \oplus g(b')\big)$. By two applications of the triangle inequality,

$$d_{\mathcal{Y}}\big(f^*(a\oplus b), f^*(a'\oplus b')\big) \;\le\; \underbrace{d_{\mathcal{Y}}\big(f^*(a \oplus b), f^*(U)\big)}_{\text{(I)}} + \underbrace{d_{\mathcal{Y}}\big(f^*(U), f^*(U')\big)}_{\text{(II)}} + \underbrace{d_{\mathcal{Y}}\big(f^*(U'), f^*(a' \oplus b')\big)}_{\text{(III)}}.$$

By Assumption 3, terms (I) and (III) are zero. For (II), Assumption 4 gives

$$d_{\mathcal{Y}}\Big(f^*(U),\, M\big(f^*(g(a)), f^*(g(b))\big)\Big) = 0, \qquad d_{\mathcal{Y}}\Big(f^*(U'),\, M\big(f^*(g(a')),\, f^*(g(b'))\big)\Big) = 0.$$

Assumption 2 and the premises yield

$$d_{\mathcal{Y}}\big(f^*(g(a)), f^*(g(a'))\big) \le d_{\mathcal{Y}}\big(f^*(g(a)), f^*(a)\big) + d_{\mathcal{Y}}\big(f^*(a), f^*(a')\big) + d_{\mathcal{Y}}\big(f^*(a'), f^*(g(a'))\big) = 0,$$

and analogously $d_{\mathcal{Y}}\big(f^*(g(b)), f^*(g(b'))\big) = 0$. The oracle-level well-definedness in Assumption 4 therefore implies

$$d_{\mathcal{Y}}\Big(M\big(f^*(g(a)), f^*(g(b))\big),\; M\big(f^*(g(a')),\, f^*(g(b'))\big)\Big) = 0,$$

which forces (II)$= 0$ by triangle inequality. Hence the whole right-hand side is zero. □

Proposition 2 (A merge law on oracle values (associative up to $d_{\mathcal{Y}}$))**.** *Define, for oracle values that actually arise on-range, the binary operation*

$$y_1 \;\odot\; y_2 \;:=\; M(y_1, y_2).$$

Then for all y_1, y_2, y_3 in the on-range set $f^*(g(\mathcal{X}))$, we have

$$d_{\mathcal{Y}}\Big((y_1 \odot y_2) \odot y_3,\; y_1 \odot (y_2 \odot y_3)\Big) = 0,$$

and $y_0 := f^*(g(\varnothing))$ is a two-sided identity in the same $d_{\mathcal{Y}}$–sense:

$$d_{\mathcal{Y}}(y_0 \odot y,\; y) = d_{\mathcal{Y}}(y \odot y_0,\; y) = 0 \quad \textit{for all on-range } y.$$

Consequently, concatenation on $\mathcal{X}$ descends, via f^*, to an associative merge on oracle values up to the task metric*.*

Proof. Pick $a, b, c \in \mathcal{X}$ with $y_1 = f^*(g(a))$, $y_2 = f^*(g(b))$, $y_3 = f^*(g(c))$. By Assumption 4,

$$(y_1 \odot y_2) \odot y_3 \;\text{ is oracle-equal to }\; f^*\big(g\big(g\big(g(a) \oplus g(b)\big) \;\oplus\; g(c)\big)\big),$$

and

$$y_1 \odot (y_2 \odot y_3) \;\text{ is oracle-equal to }\; f^*\big(g\big(g\big(a\big) \;\oplus\; g\big(g(b) \oplus g(c)\big)\big)\big).$$

Applying Assumption 3 twice yields

$$d_{\mathcal{Y}}\Big((y_1 \odot y_2) \odot y_3,\; f^*\big((a \oplus b) \oplus c\big)\Big) = 0, \qquad d_{\mathcal{Y}}\Big(y_1 \odot (y_2 \odot y_3),\; f^*\big(a \oplus (b \oplus c)\big)\Big) = 0.$$

Since concatenation on $\mathcal{X}$ is literally associative, $(a \oplus b) \oplus c = a \oplus (b \oplus c)$, hence the two targets of the displays are equal and therefore at $d_{\mathcal{Y}}$–distance zero from one another. The identity statements follow in the same way with $b = c = \varnothing$ and Assumptions 2–4. □

Remark 4 (Dropping the parent g at a merge)**.** *By Assumption 2, $d_{\mathcal{Y}}\big(f^*\big(g(g(u)\oplus g(v))\big),\, f^*\big(g(u)\oplus g(v)\big)\big) = 0$ for all u, v. Thus, in Assumption 3, one may replace the right-hand side by $f^*\big(g(u) \oplus g(v)\big)$ without changing any conclusion in the $d_{\mathcal{Y}}$–sense. We keep the explicit parent call to mirror the deployed pipeline.*

What this variant does *not* assume. We have not assumed that M is literally associative on *all* of $\mathcal{Y}$, nor that f^* embeds $\mathcal{X}$ into $\mathcal{Y}$ as a strict monoid. Propositions 1–2 assert the exact strength warranted by the applications: along on-range values produced by g, and with equalities read in the task metric, the two-route global laws make concatenation behave as an associative merge on oracle values. This global structure can be useful when the task semantics truly admit a merge law (e.g., counts); the main-text guarantees, however, require only the local, auditable laws.

G Relationship to Mergeable Summaries

This appendix formalizes the connection between our Oracle-Preserving Summarization (OPS) framework and the theory of *mergeable summaries* developed for streaming and distributed databases [Agarwal et al., 2013a]. We show that OPS strictly generalizes mergeable summaries: when the oracle f^* is symmetric and the summarizer g is deterministic, our Parentwise Compatibility law (L2) is identical to the mergeability condition. When f^* is order-sensitive or g is stochastic, OPS extends the notion of mergeability to the free monoid of strings by replacing hand-designed merge operators with learned semantic merges implemented by an LLM.

G.1 Formal Setup of Mergeable Summaries

In the classical setting, data arrives as a collection of items, and the objective is to maintain a compact synopsis that can be merged efficiently. Let $\mathcal{D}$ denote datasets (typically multisets) and $\mathcal{S}$ the sketch space with $|\mathcal{S}| \ll |\mathcal{D}|$.

- **Data:** $D_1, D_2 \in \mathcal{D}$.
- **Operation:** Multiset union $\uplus$, which is associative and commutative.
- **Summary:** A function $S : \mathcal{D} \to \mathcal{S}$.
- **Query:** Q maps a sketch to the target statistic, with distortion bounded by ϵ.

Definition 8 (Mergeable Summary)**.** *A summary S is mergeable if there exists $\mathcal{M} : \mathcal{S} \times \mathcal{S} \to \mathcal{S}$ such that for all D_1, D_2,*

$$d_{\mathcal{Y}}(Q(\mathcal{M}(S(D_1), S(D_2))),\, Q(D_1 \uplus D_2)) \leq \epsilon.$$

The guarantee holds uniformly over arbitrary merge trees, allowing sketches to be composed without revisiting the raw inputs.

G.2 Mapping OPS to Mergeable Summaries

Table 2 aligns the notation used in Section 2 with the streaming literature. The distinction is that OPS works over the free monoid of strings $(\mathcal{X}, \oplus)$, where $\oplus$ is generally non-commutative, and uses an LLM summarizer g instead of a fixed algebraic operator.

G.3 Equivalence in the Commutative Case

Suppose the task ignores order, such as counting keyword occurrences or extracting the set of unique actors. In this case, f^* acts as a homomorphism from $(\mathcal{X}, \oplus)$ to a commutative monoid $(\mathcal{Y}, +)$:

$$f^*(u \oplus v) = f^*(u) + f^*(v),$$

up to the oracle metric $d_{\mathcal{Y}}$.

Component	OPS (this paper)	Mergeable summaries
Input	Strings $u, v \in \mathcal{X}$	Datasets $D_1, D_2 \in \mathcal{D}$
Combination	Concatenation $\oplus$ (non-commutative)	Union $\uplus$ (commutative)
Summarizer	$g : \mathcal{X} \rightsquigarrow \mathcal{X}$	$S : \mathcal{D} \to \mathcal{S}$
Task/query	Oracle $f^* : \mathcal{X} \to \mathcal{Y}$	Query Q
Merge algorithm	$g(g(u) \oplus g(v))$	$\mathcal{M}(S(D_1), S(D_2))$
Consistency law	Parentwise compatibility (L2)	Mergeability condition

Table 2: Mapping OPS to the mergeable summaries framework.

Parentwise compatibility then states that the oracle applied to the merged summaries matches the oracle on the concatenated raw text:

$$\mathbb{E}\Big[d_{\mathcal{Y}}\Big(f^*\big(g(g(u) \oplus g(v))\big),\ f^*(u \oplus v)\Big)\Big] = 0.$$

When g is deterministic, the expectation drops and the condition becomes pointwise.

Proposition 3 (Reduction to Mergeable Sketches)**.** *Let f^* be commutative so that $f^*(u \oplus v) = f^*(v \oplus u)$, and let g be deterministic. If g satisfies exact Parentwise Compatibility* (L2) *on every realized merge, then g is a mergeable summary for f^* in the sense of Agarwal et al. [2013a].*

Proof. Define the merge algorithm $\mathcal{M}(s_1, s_2) = g(s_1 \oplus s_2)$, where $s_i = g(u_i)$ are summaries. For any u, v we compare two routes: (i) concatenate the raw text and apply f^*, yielding $f^*(u \oplus v)$; (ii) merge the summaries via $\mathcal{M}$ and apply f^*, yielding $f^*(g(g(u) \oplus g(v)))$. Parentwise compatibility enforces equality between these two values, so the mergeability condition holds with $\epsilon = 0$. □

Remark 5 (The universal merge)**.** *Classical sketches design bespoke $\mathcal{M}$ operators (vector addition for Count-Min, rank pruning for quantile sketches). OPS instead uses a universal merge: concatenate the summaries and apply the same summarizer again. The summarizer learns the algebra needed by the downstream oracle during its initial reduction pass, so no task-specific merge code is required.*

G.4 Generalization to Non-Commutative Tasks

The OPS framework extends mergeability to the free monoid where $u \oplus v \neq v \oplus u$. Narrative queries—"Did the protest in paragraph A trigger the crackdown in paragraph B?"—depend on order and context. Standard sketches over multisets cannot express these dependencies because the union operator discards order. Parentwise compatibility enforces a non-commutative analogue: the interaction between summarized blocks mimics the interaction between the original texts when evaluated by f^*. This is precisely the condition audited in Figure 3.

G.5 Error Propagation and Bounds

Mergeable summaries typically admit error bounds that grow logarithmically with tree height. Theorems 1 and 2 give an analogous "zero-error" propagation: if local merges satisfy Parentwise Compatibility exactly (in expectation), the global error collapses to zero regardless of tree shape. When local failures are bounded by ϵ, our deployment bound (1) plays the role of classical error accumulation: the total distortion is controlled by a weighted sum of leaf, merge, and stabilization failure rates. This is the semantic counterpart to bounding sketch errors with respect to the number of merge operations; it justifies the probabilistic audit in Section D, where random sampling over the summary tree produces a high-confidence certificate for the entire hierarchy.

H Existence of Oracle-Preserving Summaries

Two existence notions matter: set-theoretic (*some* g exists) and operational (a *short* g exists that fits a budget). We state both in terms of the task metric.

Proposition 4 (Canonical representative map in metric form)**.** *Because $\mathcal{X}$ is countable, define $x \sim x'$ iff $d_{\mathcal{Y}}(f^*(x), f^*(x')) = 0$. Fix a total order on $\mathcal{X}$ and let $c : \mathcal{Y} \to \mathcal{X}$ send y to the least x with $d_{\mathcal{Y}}(f^*(x), y) = 0$. Then*

$$g_{\mathrm{can}}(x) := c\big(f^*(x)\big) \quad \textit{satisfies} \quad d_{\mathcal{Y}}\big(f^*(g_{\mathrm{can}}(x)),\, f^*(x)\big) = 0 \ \ \textit{for all } x,$$

and g_{can} is idempotent on its range. Hence $g_{\mathrm{can}} \in \mathcal{G}^\star$ in the sense of Definition 3.

Proposition 5 (Compact encodings)**.** *If there exists an injective encoder $\mathrm{enc} : \mathcal{Y} \to \mathcal{X}$ of uniformly bounded length such that $d_{\mathcal{Y}}(f^*(\mathrm{enc}(y)), y) = 0$ for all $y \in \mathcal{Y}$, then*

$$g_{\mathrm{enc}}(x) := \mathrm{enc}(f^*(x)) \quad \textit{obeys} \quad d_{\mathcal{Y}}\big(f^*(g_{\mathrm{enc}}(x)),\, f^*(x)\big) = 0$$

and is idempotent on its range. Thus $g_{\mathrm{enc}} \in \mathcal{G}^\star$ and returns short strings.

Proposition 4 certifies internal consistency: exact, idempotent, oracle-preserving normal forms always exist; Proposition 5 records the operational version used in practice (the summary *is* a compact encoding of the oracle).

I Worked Examples and Templates

This section provides instantiation templates for four common task types, showing how to check the local laws (L1)-(L3) in practice. Each example specifies the oracle space $\mathcal{Y}$, the metric $d_{\mathcal{Y}}$, and concrete audit procedures for leaf sufficiency, parentwise compatibility, and on-range stability. The first three examples cover standard deployment scenarios (binary event detection, structured entity extraction, and numeric aggregation); the fourth is a minimal counterexample demonstrating that on-range stability (L3) cannot be omitted without breaking multi-round preservation.

I.1 Binary event detection (QA/passage-level)

- $Y = \{0, 1\}$, $d_{\mathcal{Y}}(y, y') = \mathbf{1}\{y \neq y'\}$.
- Leaf law: ask g to emit `EVENT: YES/NO` for each block; verify $\mathbb{E}[\mathbf{1}\{f^*(g(b)) \neq f^*(b)\}] = 0$ within CI.
- Parentwise: for realized u, compare $f^*(S(u_L) \oplus S(u_R))$ to $f^*(g(g(S(u_L)) \oplus g(S(u_R))))$; failures indicate cross-block evidence interactions not preserved by g.

I.2 Entity extraction (tuple)

- $Y \subseteq \mathcal{A} \times \mathcal{ACT} \times \mathcal{T} \times \mathcal{P}$ with weighted Hamming distance on fields.
- Canonicalization: require one actor/action per line in fixed order; idempotence checks use the same schema to keep g on-range.

I.3 Counting (numeric aggregation)

- $Y = \mathbb{N}$, $d_{\mathcal{Y}}(y, y') = |y - y'|$.
- Global variant instantiates $y_1 \odot y_2 = y_1 \oplus y_2$ under exact f^*-sufficiency; edgewise audits reduce to additivity checks at realized merges.

I.4 Stability Is Substantive: A Minimal Counterexample

Let $\mathcal{X}$ contain special tokens POS, NEG, and let $f^*(\text{POS}) = 1$, $f^*(\text{NEG}) = 0$, with general $f^*(x) \in \{0, 1\}$. Define

$$g(x) = \begin{cases} \text{POS}, & f^*(x) = 1,\ x \notin \{\text{POS}, \text{NEG}\}, \\ \text{NEG}, & f^*(x) = 0,\ x \notin \{\text{POS}, \text{NEG}\}, \\ \text{NEG}, & x \in \{\text{POS}, \text{NEG}\}. \end{cases}$$

Then L1 holds on leaves never equal to {POS,NEG}, but g is not on-range stable since $f^*(g(\text{POS})) = 0 \neq 1 = f^*(\text{POS})$. Thus additional normalization rounds can flip the label; L3 is required to guarantee Theorem 2.